

12-2022

LUNG CANCER TYPE CLASSIFICATION

Mohit Ramajibhai Ankoliya

Follow this and additional works at: <https://scholarworks.lib.csusb.edu/etd>



Part of the [Other Computer Engineering Commons](#)

Recommended Citation

Ankoliya, Mohit Ramajibhai, "LUNG CANCER TYPE CLASSIFICATION" (2022). *Electronic Theses, Projects, and Dissertations*. 1577.

<https://scholarworks.lib.csusb.edu/etd/1577>

This Project is brought to you for free and open access by the Office of Graduate Studies at CSUSB ScholarWorks. It has been accepted for inclusion in Electronic Theses, Projects, and Dissertations by an authorized administrator of CSUSB ScholarWorks. For more information, please contact scholarworks@csusb.edu.

LUNG CANCER TYPE CLASSIFICATION

A Project
Presented to the
Faculty of
California State University,
San Bernardino

In Partial Fulfillment
of the Requirements for the Degree
Master of Science
in
Computer Science

by
Mohit Ramajibhai Ankoliya
December 2022

LUNG CANCER TYPE CLASSIFICATION

A Project
Presented to the
Faculty of
California State University,
San Bernardino

by
Mohit Ramajibhai Ankoliya
December 2022

Approved by:

Dr. Jennifer Jin, Advisor, Computer Science and Engineering

Dr. Bilal Khan, Committee Member

Dr. Yan Zhang, Committee Member

© 2022 Mohit Ramajibhai Ankoliya

ABSTRACT

Lung cancer is the third most common cancer in the U.S. This research focuses on classifying lung cancer cells based on their tumor cell, shape, and biological traits in images automatically obtained by passing through the convolutional layers. Additionally, I classify whether the lung cell is adenocarcinoma, large cell carcinoma, squamous cell carcinoma, or normal cell carcinoma. The benefit of this classification is an accurate prognosis, leading to patients receiving proper therapy. The Lung Cancer CT(Computed Tomography) image dataset from Kaggle has been drawn with 1000 CT images of various types of lung cancer. Two state-of-the-art convolutional neural networks (CNNs) architectures, NFNets and EfficientNetB4, are trained, validated, and tested over CT-Scan images. The experiment analysis signifies that NFNets classifies lung cancer images with 96% accuracy and EfficientNetB4 with 94% accuracy in this study. With the increase in the size of the dataset, it is predicted that the accuracy will improve.

ACKNOWLEDGEMENTS

With immense pleasure, I would like to present this project report on “Lung Cancer Type Classification.” It has been an enriching experience for us to undergo such a project of Deep Learning and Machine Learning, which would not have been possible without the goodwill and support of the people around us.

As a California State University – San Bernardino student, I express deep gratitude towards Dr. Jennifer Jin, my project advisor, and committee members Dr. Bilal Khan and Dr. Yan Zhang for their support during the whole study session and development. Because of them, we were prompted to do hard work and adopt new technologies.

We are also thankful to our university for providing such self-development subjects in our curriculum, which help us explore different technologies.

TABLE OF CONTENTS

ABSTRACT.....	iii
ACKNOWLEDGEMENTS	iv
LIST OF TABLES	vii
LIST OF FIGURES	viii
CHAPTER ONE: INTRODUCTION	1
Background	1
Significance	2
Purpose	2
CHAPTER TWO: OVERALL DESCRIPTION.....	4
Hardware Requirement.....	4
Software Development Requirement.....	4
Python (Programming Language)	4
Google Colab	4
PyTorch	5
Neptune.AI	5
Streamlit	6
Dataset Information	6
Technology Stack and its Application.....	7
Machine Learning.....	8
Deep Learning.....	9
NFNets	9
EFFNet	10

Flow of the Project	11
CHAPTER THREE: METHOD	13
Image Transformation	13
Models	15
Optimizer	18
Schedule	18
Activation Function	19
Loss Function	19
CHAPTER FOUR: RESULTS	20
Train Loss	20
Train Accuracy	22
Valid Loss	24
Valid Accuracy	26
Conclusion	32
APPENDIX A: CODE OF CRUCIAL PART	33
REFERENCES	40

LIST OF TABLES

Table 1: 5-Fold Metrics for (NFNets & EFFNetB4)	28
Table 2: Test Sets Results(NFNets, BFFNetB4).....	29

LIST OF FIGURES

Figure 1: Data Set	7
Figure 2: EFFNet Model Scaling.....	11
Figure 3: Flow of the Project	12
Figure 4: Actual Image	14
Figure 5: Augmented Image.....	15
Figure 6: Efficient Net Paper, Benchmarks	16
Figure 7: NFNets Paper, Benchmarks.....	17
Figure 8: Code(important variable)	19
Figure 9: Train Loss (NFNets, EFFNetB4)	21
Figure 10: Train Accuracy (NFNets, EFFNetB4).....	23
Figure 11: Valid Loss (NFNets, EFFNetB4)	25
Figure 12: Valid Accuracy (NFNets, EFFNetB4).....	27
Figure 13: Test Sets Model Performance	30
Figure 14: Lung Cancer Type Classifier (Web Interface).....	31

CHAPTER ONE

INTRODUCTION

Background

In this project, the NSCLC (Non-small cell lung cancer) has been classified, and different experiments have been carried out. Mainly there are two types of lung cancer: an SCLC (small cell lung cancer) and an NSCLC (non-small cell lung cancer) [11]. SCLC affects 15 to 20% of people, while NSCLC affects 80% to 85% worldwide [11]. Consequently, we classified NSCLC subtypes in our experiment (adenocarcinoma, squamous cell carcinoma, and large cell carcinoma) [11].

Adenocarcinoma is the most frequent kind of lung cancer; lung adenocarcinoma accounts for around 40% of all NSCLC cases and 30% of all cases [14]. Breast, prostate, and colorectal cancers are frequent malignancies that include adenocarcinomas [14]. Lung adenocarcinomas are tumors that develop in the mucus-secreting glands that line the outside of the lung. Coughing, hoarseness, weight loss, and weakness are symptoms [14].

Squamous Cell Carcinoma is a specific variety of lung cancer that develops in the most airway branch or in the center of the lung, where the bigger bronchi connect the trachea to the lung [14]. About 30% of all non-small cell lung cancers are squamous cell lung cancers, and smoking is often a risk factor [14].

Anywhere in the lung can develop large-cell undifferentiated carcinoma lung cancer, which can develop and spread swiftly [14]. 5 to 10 percent of NSCLC patients often expect this kind of lung cancer [14]. Large-cell, undifferentiated carcinoma has a propensity for rapid growth and dispersal [14].

Significance

This lung cancer research can improve patient survival and quality of life. because they can undergo treatment priorly and their medication as well as therapy properly done. This can save the patient's life. Moreover, for people who have been diagnosed with lung cancer, research can provide a healthier and longer future, and it can also, in the end, increase the number of survivors still alive today. In the age of personalized medicine, lung cancer classification aims to link tumor cell shape to the biological traits of the tumor, making treatment choices and prognostic result prediction more accurate.

Purpose

The basic objective of this research is to achieve a higher degree of accuracy in the classification of various kinds of lung cancer. Customizing each patient's therapy for lung cancer based on precise histologic grading and biomarker data is at the heart of personalized medicine. Consequently, the histologic classification of lung cancer is becoming increasingly important in the multidisciplinary approach to diagnosing and treating lung cancer. A thorough

and precise classification of lung variations has been created to recognize the biological variety of lung cancer, which is crucial for patients' therapy and prognosis. The pathology for lung cancer has grown to include tissue diagnosis and the identification of lung cancer subtypes for further molecular analysis. Early-stage illness is surgically removed when a confirmatory histologic diagnosis is made. However, according to pathologic categorization and molecular testing, genotype-based treatment regimens and tumor types-tailored adjuvant therapy improve advanced disease survival rates.

CHAPTER TWO

OVERALL DESCRIPTION

Hardware Requirement

- Memory: 4 GB (minimum)
- Graphics Card: NVIDIA GeForce GTX 970/ Apple M1 Chip/ AMD Radeon RX 480, NVIDIA Tesla P100
- CPU: Intel Core i5 or above, Apple M series
- OS: Windows, Mac OS, Linux

Software Development Requirement

Python (Programming Language)

Python is a widely used general-purpose programming language that is dynamically typed, and garbage collected. It falls in the high-level category of programming languages, and it is extensively accepted for website development, Software development, Automation, and Data Analytics. Python frameworks for different purposes like Django, Flask for Web Development, and SciPy, Pandas, NumPy, TensorFlow, Keras, and PyTorch for Machine Learning.

Google Colab

Google Colab is a cloud-based platform to train Machine Learning models is utilized. With the more extensive neural networks, graphical-based processing (GPUs) is required that can accelerate the model training. Lung cancer type

classification via images requires efficient matrix operations, for which GPU is preferred. Thus, to train the deep learning models for lung cancer type classification, Google Colab is utilized. Moreover, it can maintain the versioning and is helpful as a version control tool.

PyTorch

PyTorch is an open-source machine learning framework based on the torch library and used in NLP (Natural Language Processing) and Computer Vision applications. The framework offers libraries, namely, Torch Text, Torch Vision, and Torch Audio, all domain specific. In PyTorch, tensors are used to encode the input and output of a model and the model's parameters. Pytorch allows us to use pre-loaded datasets along with our data to separate our dataset from our model training code for better interpretation and modularity. The two offered primitives include Data Loader and Dataset. This dataset is used for storing samples and the respective labels. At the same time, Data Loader wraps an iterable around the datasets for easy access to samples. PyTorch also offers "transforms," which can be used to transform and manipulate data to prepare it for training.

Neptune.AI

Neptune.AI is a lightweight experiment management MLOps solution that can be used for multiple purposes covering data exploration notebooks and experiments. We can use it as a service or deploy it on the cloud or hardware. Neptune integration consists of over 25 Python, Artificial Intelligence, or Machine

Learning libraries, including Matplotlib, Pyplot, PyTorch, TensorFlow, Keras, Scikit-learn, and many more. It also allows experiments typed in more than one language like R, Python, or any different language and Notebook types, such as Jupyter, Deep note, Google Colab, and Amazon Sage Maker. Neptune.AI is an effective alternative to visualize, store, organize and compare metadata inclined with an AI/ML project.

Streamlit

A web interface was created using Streamlit to make the application user-friendly and useful for doctors, where they can upload the CT scan image, and the application automatically tells the type of lung cancer. It is an Open-Source application framework in Python compatible with major Python libraries like Scikit-learn, Keras, PyTorch, SymPy(latex), NumPy, Pandas, and Matplotlib.

Dataset Information

Lung CT-Scan images Dataset contains 1000 CT-Scan images with different types of chest cancer [4]. Images have two other formats, a total of 988 images have a .png format, and another 12 images have a .jpg format [4].

Figure 1 shows the type of lung cancers and their sample images.

The dataset is divided into four classes:

- Adenocarcinoma
- Large cell carcinomas
- Squamous cell carcinomas

- Normal cells

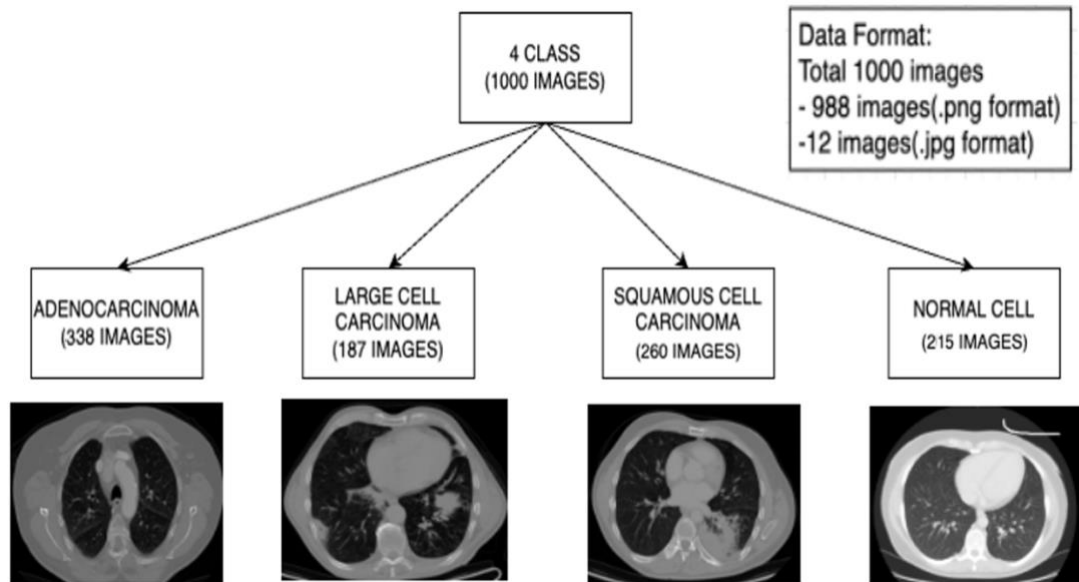


Figure 1: Data Set

Chest CT-Scan images Dataset. (n.d.). Retrieved November 6, 2022, from <https://www.kaggle.com/datasets/mohamedhanyyy/chest-ctscan-images>

Technology Stack and its Application

This chapter has mentioned the general background behind the technology used in this project for image classification. This image classification methodology is based on machine learning and deep learning techniques such as NFNNets for rapid training and good performance.

Machine Learning

Machine Learning and Symbolic Learning are two major fields of Artificial Intelligence. Deep Learning is a Machine Learning method based on Artificial neural networks and pattern recognition. One of the applications of Deep Learning is in the field of computer vision. In deep learning, the tasks are carried out without explicit knowledge and rely only on prior patterns and inference, also known as computational intelligence. It serves as the basis for collecting statistical tools that may estimate complex functions through learning from accumulated data. It can be further broken down into three primary components related to training the model: supervised learning, unsupervised learning, and Reinforcement Learning. During the training phase of supervised learning, the computer is provided with input data and is expected to produce the correct output.

Reinforcement learning contrasts with the previous two techniques in that it does not need the labeled input/output pairings or the knowledge of patterns. Instead, the emphasis is on striking a balance between the investigation of the previously unexplored region and the utilization of current knowledge. Machine learning is used in various fields such as Natural Language Processing (like sentiment analysis, text classification, etc.), Speech Processing (like Speech Recognition, Speech Enhancement, Speaker Diarization, etc.), Computer Vision and Image Processing (like Medical Image Classification, Object detection, etc.).

Briefly, Machine Learning is the study of how to make computers learn without being explicitly taught and how these machines may make decisions with the help of what they know about people.

Deep Learning

The term "Deep Learning" describes a group of machine learning algorithms that can extract more abstract features, often from raw input on a progressive basis, using a framework of Artificial Neural Networks to include numerous layers. Deep Learning is a subset of Machine Learning.

A massive portion of computational power is needed for deep learning. GPUs with a high level of performance have a parallel design, which makes them efficient for deep learning. This fact enables software developers to cut the training time for a deep learning network from weeks to hours or fewer when paired with clusters or cloud computing. Previously, the training period was weeks. Deep learning is an essential part of the technology enabling self-driving automobiles to identify a stop sign and differentiate between a pedestrian and another vehicle. Deep learning is now receiving a lot of attention, and there is a strong explanation for this trend. It is the accomplishment of outstanding outcomes that were not attainable in the past.

NFNets

NFNets is a short form for Normalizer-Free Networks. This fact is the image classification technique that does not need batch normalization and is a faster technique than other methods that use batch normalization [3].

NFNets are a family of modified ResNets that achieves competitive accuracies without batch normalization [3]. To do so, it applies three different techniques:

- Modified residual branches and convolutions with Scaled Weight Standardization
- Adaptive Gradient Clipping
- Architecture optimization for improved accuracy and training speed

EFFNet

Figure 2 represents EFFNet model along with its scaling like baseline, width scaling, deep scaling, resolution scaling, and compound scaling which are the 3d space zooming methodology implemented to improve the accuracy of deep learning classification model [10]. In the past, Convolutional Neural Networks were often constructed on a set resource budget, and the level of accuracy might be scaled up for improved performance if additional resources were made available [10]. Then EFFNet is a family of models designed on neural architecture search design to scale up and achieve better accuracy and efficiency than previous CNNs [10].

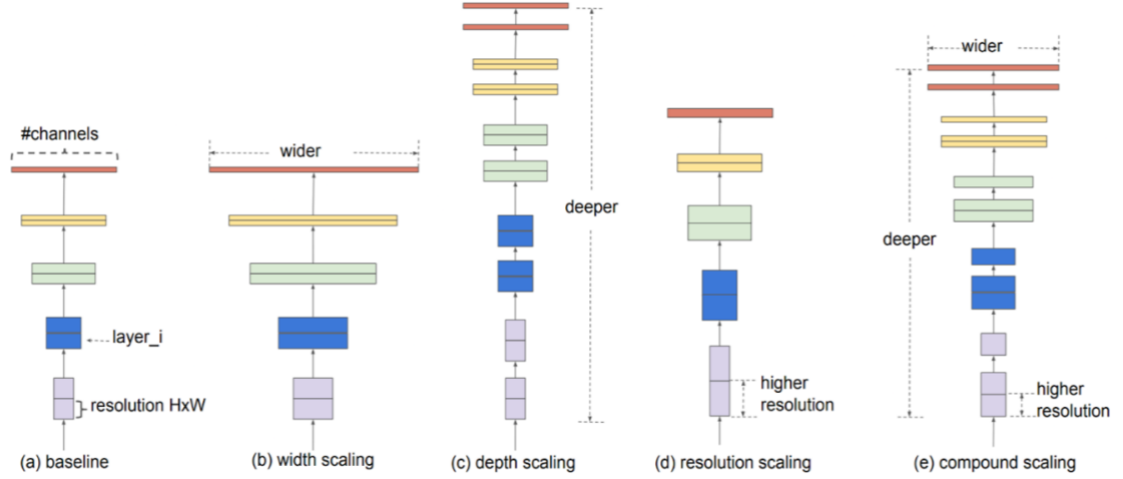


Figure 2: EFFNet Model Scaling

Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (arXiv:1905.11946).

Flow of the Project

Figure 3 shows that the original images are augmented using image transformation functions. After heavy augmentations, images and label pairs are sent as input to the CNN models, and the model outputs the predicted labels. Furthermore, the cross-entropy loss is calculated between true labels and predicted labels. After that, models' weights are updated using an Optimizer such as Ranger to minimize the loss. The training process goes on until the given epochs.

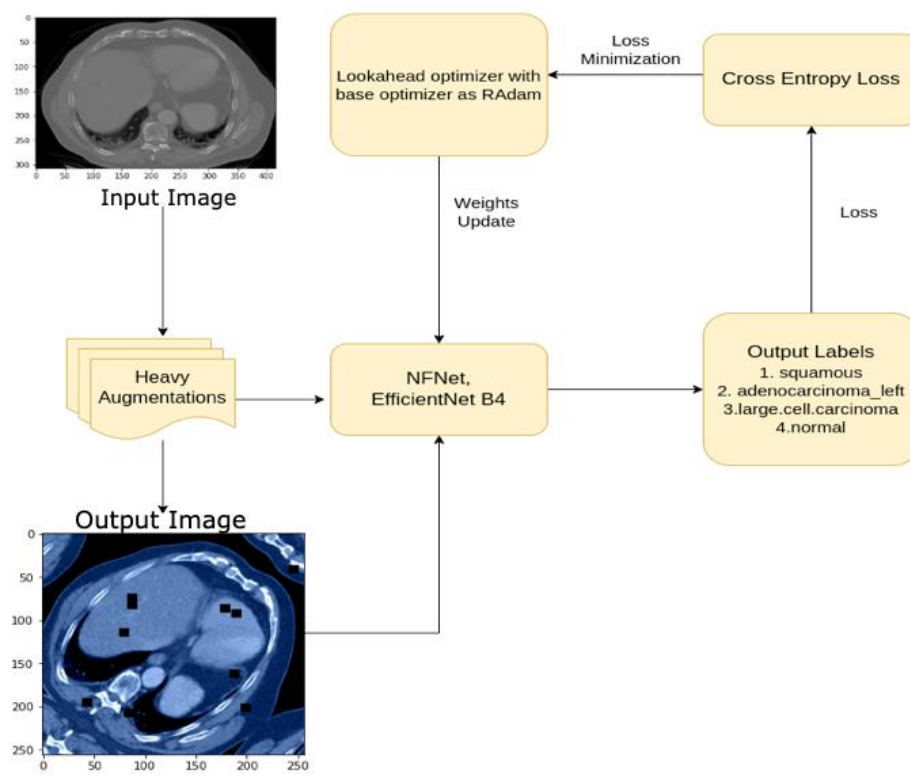


Figure 3: Flow of the Project

CHAPTER THREE

METHOD

Image Transformation

In the Image Transformation, two functions were created for the training and validation; for the training dataset, image transforms like (Resize, Horizontal Flip, Vertical Flip, Rotate, Random Brightness, Cutout, and Normalize). Each image was resized by [256, 256], Flipping transformation with a probability of 50%, Random Rotation with 50 degrees, Cutout with several holes 12 where each hole is of [10, 10], and Normalization mean, and standard deviation value was taken from ImageNet models. For the validation, dataset transforms such as Resize and Normalization were used. Using this augmentation, the aim was to create a variety and increment in the features. Before the augmentation, the image looks like Figure 4.

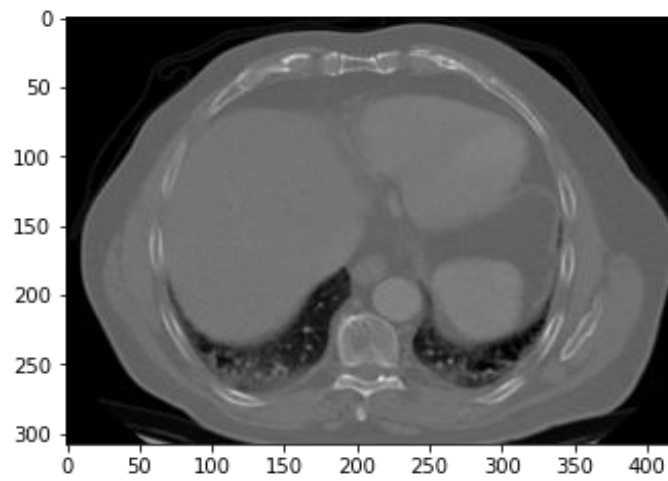


Figure 4: Actual Image

After applying the augmentation method to the dataset, the image looks as shown in Figure 5.

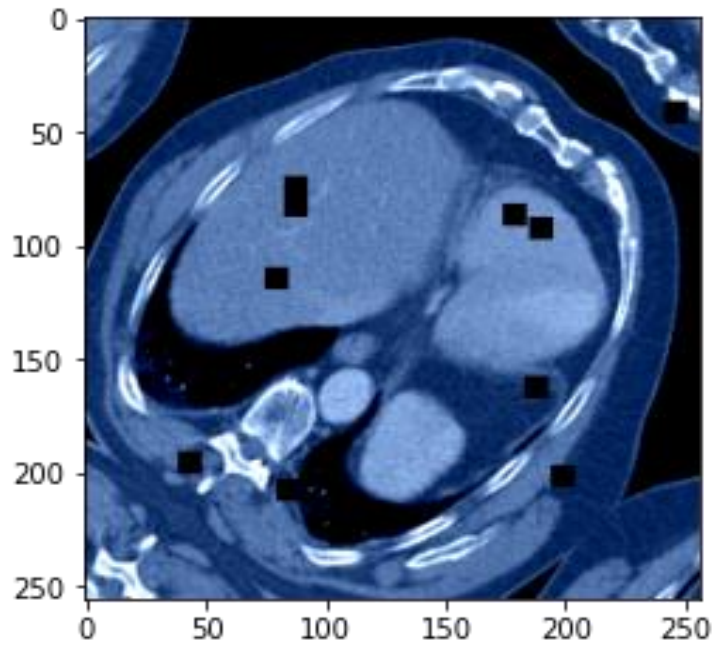


Figure 5: Augmented Image

Models

Here in this project, two state-of-the-art CNN architectures are used. The first is NfNets (Normalizer free network) and EfficientNetB4. The aim was to use one model without any batch normalization like NfNets by DeepMinds and with Batch normalization and break the benchmark results as Efficient Net. Also, the reason for using one Normalizer Free Network is that Normalizer-Free models attain significantly better performance than their batch-normalized counterparts when fine-tuning ImageNet.

Model size(Millions) vs. ImageNet Accuracy(%). In Figure 6, all Numbers are provided for a single model and crop. Efficient Nets outperformed all other

ConveNets. Our Algo is more efficient than others, given the small test case [10].

In Efficient Nets, the best performance is given by EfficientNet-B7, which is 84.3%, comes in 7.8M parameters, is 6.1x faster, and is 8.4x smaller than Gpipe. lowest accuracy was got by EfficientNet-B1 which is 79.1% , which is 7.6x smaller and 5.7x faster than ResNet-152 [10].

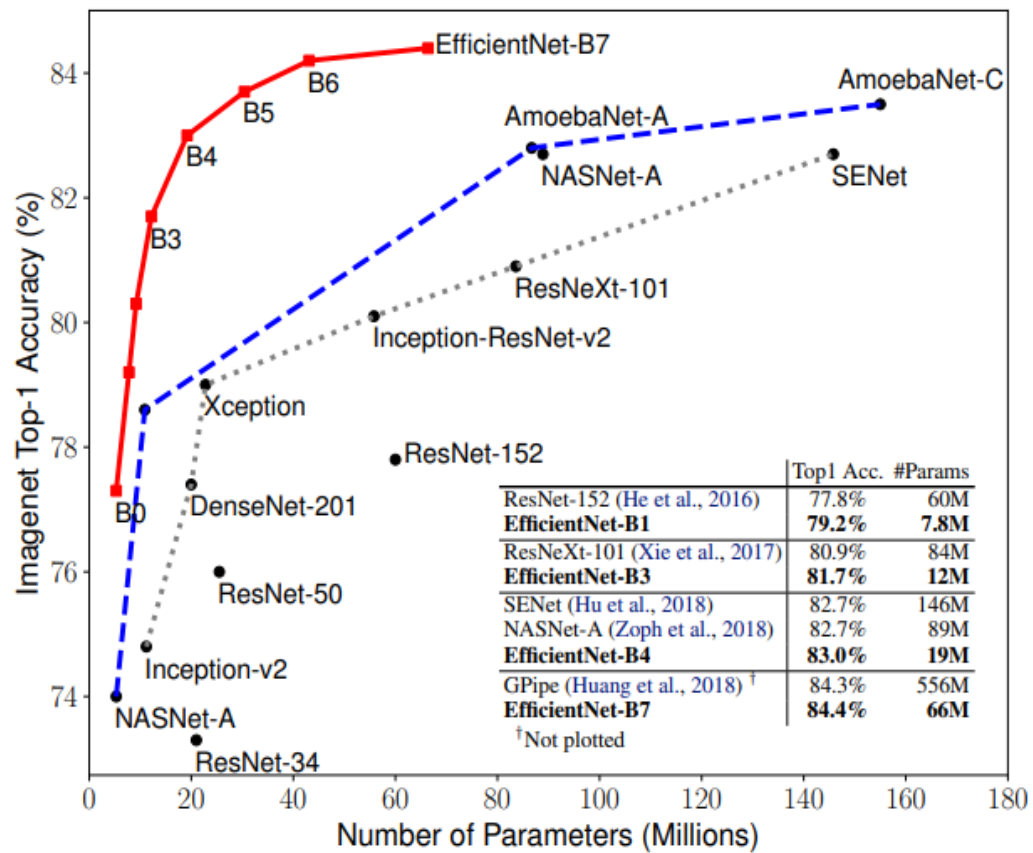


Figure 6: Efficient Net Paper, Benchmarks

Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (arXiv:1905.11946).

According to Figure 7, LambdaNet-152 achieves 83% accuracy, while LambdaNet-420 achieves 85% accuracy [3]. In addition, EFFNet-b7 achieves only 85% accuracy while being 1.1x faster than effete-b2 (80% accuracy) [3]. However, NFNet got the highest accuracy compared to other models like NFNet-f0, achieving 83.6% accuracy and infant-f5 is on top with an accuracy of 86% [3].

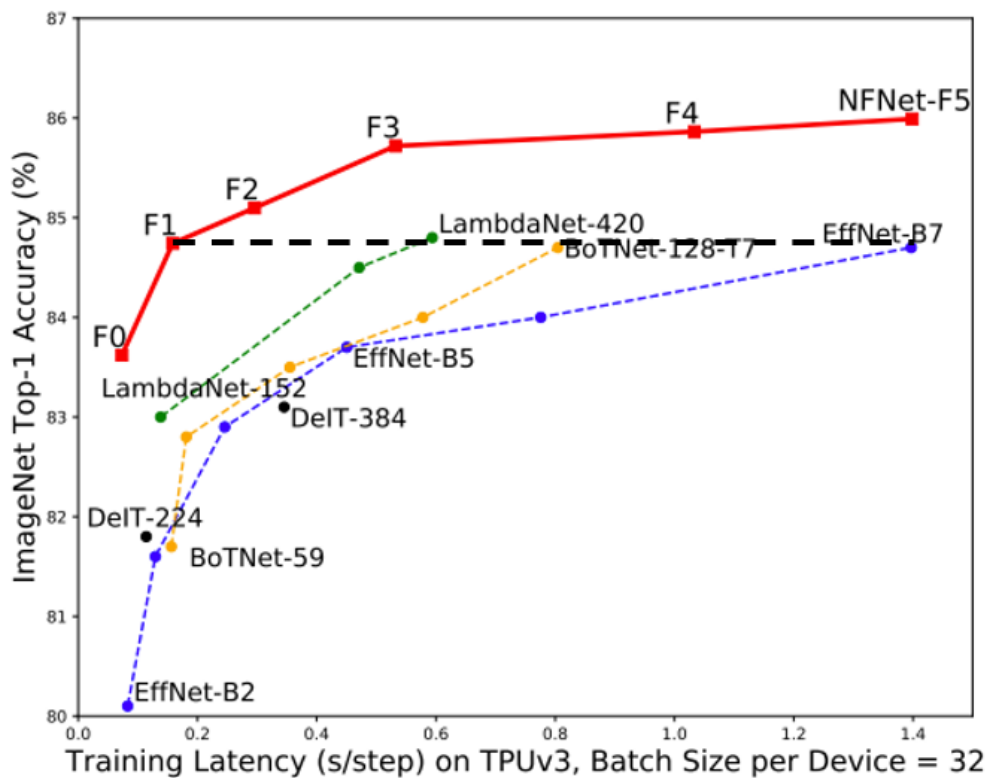


Figure 7: NFNet Paper, Benchmarks

Brock, A., De, S., Smith, S. L., & Simonyan, K. (2021). *High-Performance Large-Scale Image Recognition Without Normalization* (arXiv:2102.06171).

Optimizer

Generally, the widely used optimizer is Adam Optimizer, SGD Optimizer, and RMSProp, but here, we have used the new state-of-the-art optimizer called Ranger. Ranger combines two optimizers; the first is Lookahead Optimizer, which takes the base optimizer as RAdam. Also, gradient centralization is added to the Ranger Optimizer. The learning rate is 0.003 (differs by Alpha = 0.5.) [13].

SWA (Stochastically Weighted Averaged) was used after several epochs where the scheduler paused and averaged the weights after given epochs [13]. The SWA technique was dropped because it did not show any significant results.

Schedule

To ensure the training does not overfit, cosine annealing warm restarts were used in which the settings are $T_0 = 5$, which means at every five epochs, it will restart with the initial learning rate 0.003 and the minimum learning rate decreasing to $1e-5$ ($\eta_{\min}$). Figure 8 represents the important variable for the schedule.

```
CosineAnnealingWarmRestarts(
```

```
    T_0 = 5,
```

```
    eta_min = 1e-5,
```

```
    T_mult=0,
```

```
)
```

Figure 8: Code(important variable)

Activation Function

Generally, Models use ReLU or SiLU activation nowadays. Here, ReLU or SiLU activation function was replaced by the Mish activation function. In terms of computations, Mish is GPU hungry, but the result Mish has given in ReLU, or SiLU activation, is significantly more.

Loss Function

Here, the problem was classification, and there were four different classes; CrossEntropyLoss() function was used. I also tried Focal Loss, but it was not giving any significant results; the traditional loss function was used.

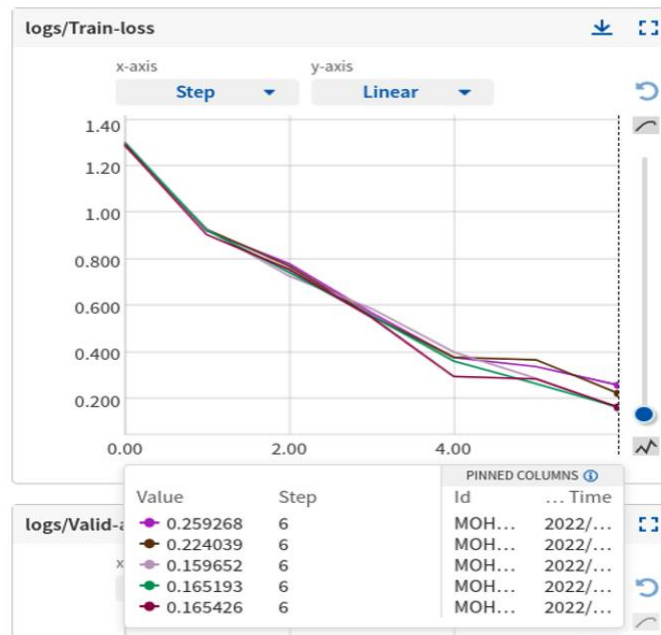
CHAPTER FOUR

RESULTS

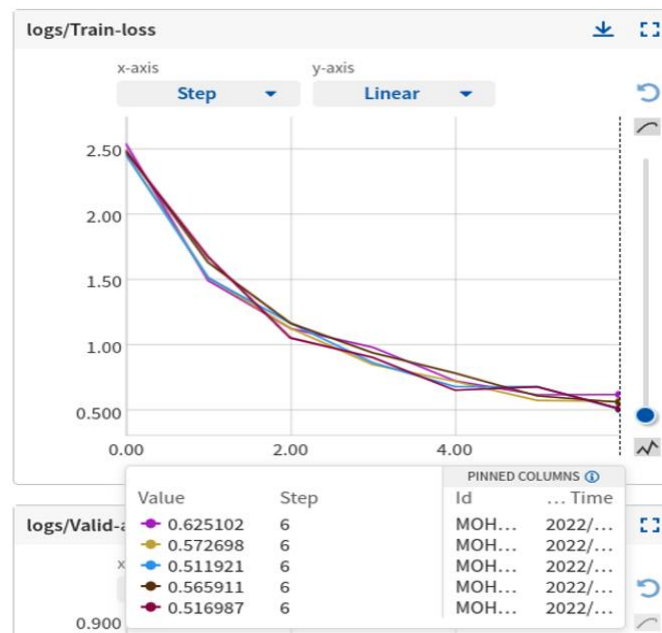
Initially, NFNets and Efficientnetb4 with five-fold cross-validation were trained to get perfect cross-validation scores and received the precision, recall, accuracy, and loss from test data sets. Neptune.AI was utilized to monitor the training and validation accuracy. The results are as follows:

Train Loss

Train Loss evaluates the error of the developed model on the input of the training data set. The output of train loss is a number, providing insight into the created model score and how it predicts the bad prediction. Internally, the calculation takes the summation of the error of each input. Figure 9 depicts the training Loss where the x-axis represents the number of epochs, and the y-axis represents the loss calculated for each epoch.



Train Loss(NFNet)



Train Loss(BFFNetB4)

Figure 9: Train Loss (NFNet, EFFNetB4)

x-axis: Number of epochs; y-axis: Loss for each epoch

Train Accuracy

The term accuracy refers to correctness; hence train accuracy is defined as the number of classification corrections while we pass the input training data to the model. Technically when it developed, it could be deducible to train the model's accuracy on input data. Figure 10 shows the training Accuracy where, the x-axis represents the number of epochs, and the y-axis represents the accuracy calculated for each epoch.



Train Accuracy(NFNet)



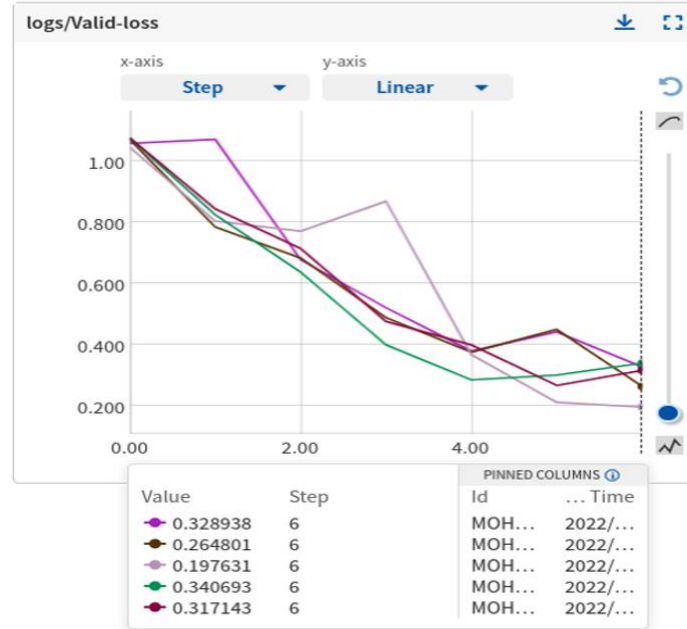
Train Accuracy(BFFNetB4)

Figure 10: Train Accuracy (NFNet, EFFNetB4)

x-axis: Number of epochs; y-axis: Accuracy for each epoch

Valid Loss

Valid Loss is defined vice versa than train loss. It is determined as Loss, a used metric to evaluate the efficiency of the developed Deep Learning or Machine Learning model on the given validation data set (it is data set away for validating the execution of the model). Validation loss is computed similarly to train Loss by summation of the error of each example. Figure 11 depicts the valid Loss where the x-axis represents the number of epochs, and the y-axis represents the Loss calculated for each epoch.



Valid Loss(NFNets)



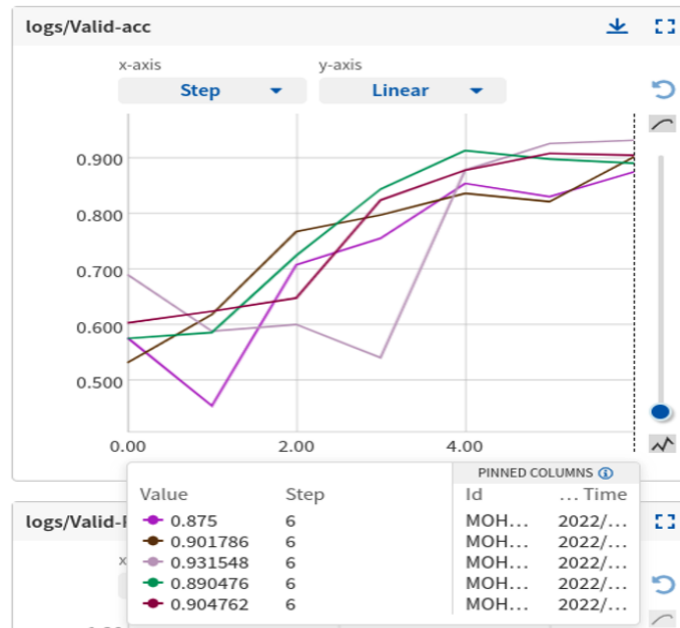
Valid Loss(EFFNetB4)

Figure 11: Valid Loss (NFNet, EFFNetB4)

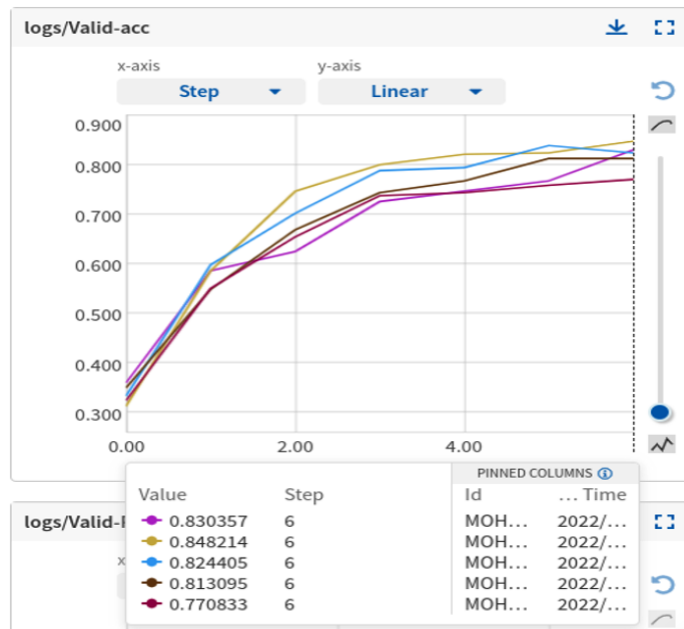
x-axis: Number of epochs; y-axis: Loss for each epoch

Valid Accuracy

Valid accuracy specifies to what extent a method or model accurately intended what it measures. For example, any research measures high validity, which causes an evaluation that leads to real qualities. The graph in Figure 12 shows the valid accuracy where the x-axis represents the number of epochs, and the y-axis represents the accuracy calculated for each epoch.



Valid Accuracy(NFNets)



Valid Accuracy(BFFNetB4)

Figure 12: Valid Accuracy (NFNets, EFFNetB4)

x-axis: Number of epochs; y-axis: Accuracy for each epoch

As per the above, there is no sign of overfitting. Also, according to model metric comparison, we can see that NFNET outperforms EfficientNetsB4 in 5-Fold CV Score and Loss. According to Table 1, the highest accuracy achieved from NFNets is more than 90%.

Table 1: 5-Fold Metrics for (NFNets & EFFNetB4)

Model	Fold	Train Loss	Train Acc	Valid Loss	Valid Acc
NFNets	Fold 1	0.259	93.75 %	0.32	87.57%
	Fold 2	0.224	93.19 %	0.26	90.17 %
	Fold 3	0.159	95.13 %	0.19	93.15%
	Fold 4	0.165	93.19 %	0.34	89.04 %
	Fold 5	0.165	92.36 %	0.31	90.47 %
	Average	0.1944	93.52 %	0.284	90.08 %
EFFNetB4	Fold 1	0.625	78.85 %	0.635	83.03 %
	Fold 2	0.572	80.64 %	0.472	84.82 %
	Fold 3	0.511	79.59 %	0.487	82.44 %
	Fold 4	0.565	79.62 %	0.531	81.30 %
	Fold 5	0.516	82.59 %	0.600	77.08 %
	Average	0.557	80.25 %	0.545	81.73 %

As per Table 2, the NFNets has better accuracy and recall score than the EFFNetB4. That clearly shows that NFNets are the best suitable for lung cancer classification. The precision is computed as the ratio of Positive samples that were correctly classified to all samples that were classified as Positive (either correctly or incorrectly) ,and the recall is determined as the proportion of Positive samples identified correctly as Positive to all Positive samples.

Table 2: Test Sets Results(NFNets, BFFNetB4)

MODEL	Accuracy	Loss	Precision	Recall
NFNets	96 %	0.1481	99.99 %	96 %
EFFNetB4	94 %	0.2007	99.99 %	93 %

Figure 13 illustrates the accuracy, precision, and recall graph for NFNets and EFFNetsB4, which shows that test sets model performance.

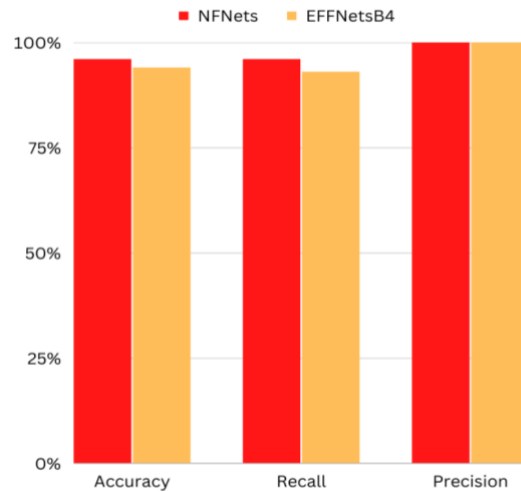


Figure 13: Test Sets Model Performance

x-axis: Accuracy, Recall, Precision; y-axis: Percentage (%)

Figure 14 shows the web interface for lung cancer type classification, and after uploading the image, this classifier shows which type of cancer there is in the test image.

LUNG CANCER TYPE CLASSIFICATION

Choose a file

Drag and drop file here
Limit 200MB per file

Browse files

000115 (4).png 106.6KB

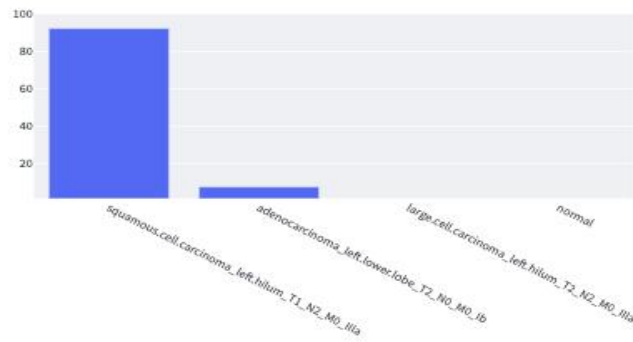


Figure 14: Lung Cancer Type Classifier (Web Interface)

Conclusion

In this Project, lung cancer cell classification has been carried out. The proposed approach relies on Computer Vision, Machine Learning, and Deep Learning and automatically talks about lung cells. The NFNets and EfficientNetB4 were trained, and it was discovered that NFNets are more accurate in classification and surpass EfficientNetB4. The NFNets are achieving 96% accuracy, whereas the EfficientNetB4 is reaching 94%. By such classification, proper therapy and early cancer treatment can be carried out. As the data set gets more significant with the addition of more data from participating patients, the model will be more efficient & accurate for prediction.

APPENDIX A
CODE OF CRUCIAL PART

Sample of Parameter

```
params = {
    'DATA_DIR' : '../input/mohit-project-ct-scan/Data',
    'CSV' : '../input/mohit-project-ct-scan/train_folds.csv',

    'SEED' : 1620,

    'MEAN' : [0.485, 0.456, 0.406],
    'STD' : [0.229, 0.224, 0.225],

    'FOLD' : 4,
    'DEVICE' : 'cuda',
    'SWA' : False,
    'SAM' : False,
    'FP16' : False,

    'IMAGE_SIZE' : 256,

    'SHUFFLE' : True,
    'PIN_MEMORY' : True,
    'NUM_WORKERS' : 2,
    'BATCH_SIZE' : 24,

    'LR' : 0.001,
    'EPOCHS' : 7,
    'T_MAX' : 7,
    'ETA_MIN' : 3e-4,

    'MODEL_NAME' : 'tf_efficientnet_b4',
    'MODEL_PATH' : './',
    'PRETRAINED' : True,
    'NUM_CLASSES' : 4,
    'IN_CHANNELS' : 3
}
```

Image Transformation using Albumentation

It is a high-speed and elastic image augmentation python library and a computer vision tool that enhances the efficiency of deep convolutional networks. It is popular in the industry for DL (Deep Learning) research, ML (Machine Learning) competitions, and open-source projects. It employs a diverse range of image transform techniques optimized for better execution. It offers a dynamic image augmentation interface for different computer vision operations, ranging from cover classification to segmentation, segmentation, and detection. In various areas, such as satellite imagery, manufacturing, and industrial applications, Generative Adversarial Networks can apply Albumentation [1]. It is compatible with deep learning frameworks like PyTorch and Keras.

```
def get_train_transforms():
    return albumentations.Compose([
        albumentations.Resize(params['IMAGE_SIZE'],params['IMAGE_SIZE'],always_apply=True),
        albumentations.HorizontalFlip(p=0.5),
        albumentations.VerticalFlip(p=0.5),
        albumentations.Rotate(limit=50, p=0.8),
        albumentations.RandomBrightness(limit=(0.09, 0.6), p=0.5),
        albumentations.Cutout(num_holes = 12, max_h_size=10, max_w_size=10, fill_value=0,
always_apply=False, p=0.5),
        albumentations.Normalize(mean = params['MEAN'], std = params['STD']),
        ToTensorV2(p=1.0),
    ])

def get_valid_transforms():
    return albumentations.Compose([
        albumentations.Resize(params['IMAGE_SIZE'],params['IMAGE_SIZE'],always_apply=True),
        albumentations.Normalize(mean = params['MEAN'], std = params['STD']),
        ToTensorV2(p=1.0),
    ])
```

Mish Function

Mish function is known as a self-regularized non-monotonic function activation function. Mathematically, it determines as $f(x) = x \tanh(\text{softplus}(x))$ [6]. It plays a crucial role in performance and efficiency while training, and it follows the self-gating property of Swish. Compared to Swish, ReLU, and Leaky ReLU, the Mish function is more accurate for improving the performance of neural network architecture and better empirical results [6]. Swish is not focused on singularities, so this is one of the drawbacks to performing gradient-based optimization.

```
class Mish_func(torch.autograd.Function):

    @staticmethod
    def forward(ctx, i):
        result = i * torch.tanh(F.softplus(i))
        ctx.save_for_backward(i)
        return result

    @staticmethod
    def backward(ctx, grad_output):
        i = ctx.saved_tensors[0]

        v = 1. + i.exp()
        h = v.log()
        grad_gh = 1./h.cosh().pow_(2)

        grad_hx = i.sigmoid()

        grad_gx = grad_gh * grad_hx

        grad_f = torch.tanh(F.softplus(i)) + i * grad_gx

        return grad_output * grad_f

class Mish(nn.Module):
    def __init__(self, **kwargs):
        super().__init__()
        print("Mish initialized")
        pass
    def forward(self, input_tensor):
        return Mish_func.apply(input_tensor)
```

The PyTorch model class, which used Timm Packages

Timm is a library of deep learning, a collection of layers, utilities, computer vision, deep-learning models, optimizers, data loaders, schedulers, and augmentations used for training and validation scripts to build ImageNet training results. It is a straightforward procedure by Timm to create a model.

```
class AutoModel(nn.Module):  
  
    def __init__(self, model_name, pretrained, num_classes, in_channels):  
        super(AutoModel, self).__init__()  
  
        self.backbone = timm.create_model(  
            model_name = model_name,  
            pretrained = pretrained,  
            num_classes = num_classes,  
            in_chans = in_channels  
        )  
  
    def forward(self, images, labels = None):  
  
        logits = self.backbone(images)  
  
        if labels != None:  
            loss = nn.CrossEntropyLoss()(logits, labels)  
            return logits, loss  
  
        return logits
```

Training Pipeline

In this training pipeline, I used Ranger optimization with FastAI, which has a single codebase for simplicity of use and efficiency (load/save and one loop handling for all param updates) and is immediately usable [13]. We employed Ranger as an optimizer to surpass the top results across 12 distinct FastAI leaderboard categories. However, strongly advise pairing Ranger with the flat+cosine anneal training curve and Mish activation function[13].

```

def seed_everything(seed=params['SEED']):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True

def run():

    seed_everything()
    df = pd.read_csv(params['CSV'])

    train_df = df[df['kfold'] != params['FOLD']]
    valid_df = df[df['kfold'] == params['FOLD']]

    trainset = CTScanDataset(train_df, get_train_transforms())
    validset = CTScanDataset(valid_df, get_valid_transforms())

    trainloader = torch.utils.data.DataLoader(
        trainset,
        batch_size = params['BATCH_SIZE'],
        shuffle = True,
    )

    validloader = torch.utils.data.DataLoader(
        validset,
        batch_size = params['BATCH_SIZE']
    )

    model = AutoModel(
        model_name = params['MODEL_NAME'],
        pretrained = params['PRETRAINED'],
        num_classes = params['NUM_CLASSES'],
        in_channels = params['IN_CHANNELS']
    )

    model = model.to(params['DEVICE'])
    model = replace_activations(model, nn.SiLU, Mish())

    optimizer = Ranger(model.parameters(), lr = params['LR'])
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer,
        T_max = params['T_MAX'],
        eta_min = params['ETA_MIN']
    )

    best_score = np.Inf

    for i in range(params['EPOCHS']):
        tr_loss, tr_acc = train_fn(model, trainloader, optimizer, scheduler, i, swa_model = None)
        vl_loss, vl_acc = eval_fn(model, validloader, i)

        if vl_loss < best_score:
            save_utils = {
                'model' : model.state_dict(),
                'best_score' : best_score,
                'epochs' : i+1,
                'optimizer_state_dict' : optimizer.state_dict(),
                'scheduler' : scheduler.state_dict()
            }
            torch.save(save_utils, params['MODEL_PATH'] + 'CTSCAN-' + params['MODEL_NAME'] + '-fold-' +
str(params['FOLD']) + '.pt')
            best_score = vl_loss
            print('SAVED-MODEL')

    run()

```

REFERENCES

- [1] *Albumentations*. (n.d.). Retrieved November 6, 2022, from <https://albumentations.ai/>
- [2] Bhattacharjee, A., Richards, W. G., Staunton, J., Li, C., Monti, S., Vasa, P., Ladd, C., Beheshti, J., Bueno, R., Gillette, M., Loda, M., Weber, G., Mark, E. J., Lander, E. S., Wong, W., Johnson, B. E., Golub, T. R., Sugarbaker, D. J., & Meyerson, M. (2001). Classification of human lung carcinomas by mRNA expression profiling reveals distinct adenocarcinoma subclasses. *Proceedings of the National Academy of Sciences of the United States of America*, 98(24), 13790–13795. <https://doi.org/10.1073/pnas.191502998>
- [3] Brock, A., De, S., Smith, S. L., & Simonyan, K. (2021). *High-Performance Large-Scale Image Recognition Without Normalization* (arXiv:2102.06171). arXiv. <http://arxiv.org/abs/2102.06171>
- [4] *Chest CT-Scan images Dataset*. (n.d.). Retrieved November 6, 2022, from <https://www.kaggle.com/datasets/mohamedhanyyy/chest-ctscan-images>
- [5] Hongwei, Y. (2022). *Gradient Centralization* [Python]. <https://github.com/Yonghongwei/Gradient-Centralization> (Original work published 2020)
- [6] Misra, D. (2020). *Mish: A Self Regularized Non-Monotonic Activation Function* (arXiv:1908.08681). arXiv. <http://arxiv.org/abs/1908.08681>
- [7] *Pathology of lung malignancies—UpToDate*. (n.d.). Retrieved November 6, 2022, from <https://www.uptodate.com/contents/pathology-of-lung-malignancies>

- [8] *Prediction and Classification of Lung Cancer Using Machine Learning Techniques—IOPscience*. (n.d.). Retrieved November 6, 2022, from <https://iopscience.iop.org/article/10.1088/1757-899X/1099/1/012059>
- [9] Sabbula, B. R., & Anjum, F. (2022). Squamous Cell Lung Cancer. In *StatPearls*. StatPearls Publishing.
<http://www.ncbi.nlm.nih.gov/books/NBK564510/>
- [10] Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (arXiv:1905.11946). arXiv.
<http://arxiv.org/abs/1905.11946>
- [11] *Types of lung cancer | Cancer Research UK*. (n.d.). Retrieved November 6, 2022, from <https://www.cancerresearchuk.org/about-cancer/lung-cancer/stages-types-grades/types>
- [12] Wang, S., Dong, L., Wang, X., & Wang, X. (2020). Classification of Pathological Types of Lungs Cancer from CT Images by Deep Residual Neural Networks with Transfer Learning Strategy. *Open Medicine*, 15, 190–197. <https://doi.org/10.1515/med-2020-0028>
- [13] Wright, L. (2022). *Ranger-Deep-Learning-Optimizer* [Python].
<https://github.com/lessw2020/Ranger-Deep-Learning-Optimizer> (Original work published 2019)
- [14] *How common is non-small cell lung cancer? | MedicalNewsToday* October 18, 2021, from <https://www.medicalnewstoday.com/articles/prevalence-of-non-small-cell-lung-cancer#summary>